

Hash Tables

C++ · Explanation & Theory Notes

Data Structures & Algorithms · Study Reference

Lookup $O(1)$

Insert $O(1)$

Delete $O(1)$

Worst $O(n)$

1 The Need for Speed — Problem & Solution

The Problem — Linear Search

When searching through massive collections of data — such as millions of phone contacts — checking items one by one via a **linear search** is painfully slow. For 1,000,000 entries this creates a significant performance bottleneck. The time grows proportionally with the dataset size: **$O(n)$** .

The Solution — Hash Table

Hash tables *skip the line entirely*. They are engineered to look up, add, or delete items in **$O(1)$ constant time**. This means finding an item takes the same amount of time whether your dataset has 10 items or 10 million — the size of the haystack simply does not matter.

Method	Time Complexity	Scales with n ?
Linear Search	$O(n)$	Yes — gets slower
Hash Table	$O(1)$	No — always instant

2 The Secret Sauce — The Hash Function

Think of a hash function as a super-efficient warehouse receptionist. When you hand it a **key** (e.g. the string "Apple"), it does not search through shelves — it instantly *calculates* the exact storage locker (**bucket**) where that item belongs.

The 4-Step Flow

1	Provide a key You supply the key — e.g. a string, integer, or object.
2	Hash function processes it The function applies a mathematical algorithm to the key bytes.
3	Numeric index generated A unique numeric index is produced, pointing to a specific bucket.
4	Instant store / retrieve Your value is stored at — or fetched from — that exact bucket position.

3 The Collision Conundrum

A **collision** occurs when a hash function maps two completely different keys (e.g. "apple" and "banana") to the *exact same bucket*. This is **not a bug** — it is an expected, mathematically inevitable consequence of mapping a near-infinite key space onto a finite number of buckets.

C++ Collision-Handling Strategies

■ Chaining	↔ Open Addressing
Each bucket acts as its own miniature linked list . When a collision occurs the new item is simply appended to that bucket's chain.	If the target bucket is already occupied, the system scans sequentially for the very next available open slot in the underlying array.
<code>bucket[i] → [v1] → [v2] → null</code>	<code>probe: i, i+1, i+2, ...</code>

4 Implementation in C++ — `std::map` vs `std::unordered_map`

The C++ Standard Library provides two primary key-value containers. Your choice depends on whether you prioritise **speed** or **sorted order**:

Feature	<code>std::map</code>	<code>std::unordered_map</code>
Underlying Structure	Balanced BST (Red-Black Tree)	Hash Table
Avg Time Complexity	$O(\log n)$	$O(1)$
Data Order	Sorted by key	Unordered
Best Used For...	When sorted key order is required	When raw speed is the top priority
Header Include	<code>#include</code>	<code>#include</code>

Quick C++ Usage

```
#include <unordered_map> std::unordered_map<std::string, int> scores;
scores["Alice"] = 95; // O(1) insert int s = scores["Alice"]; // O(1) lookup
scores.erase("Alice"); // O(1) delete
```

■ Worst-Case $O(n)$ Note

An `unordered_map` can theoretically degrade to $O(n)$ if a poor hash function forces every key into the same bucket — turning the table into a plain linked list. In practice, C++'s built-in hash functions are so well-designed that this scenario is essentially never seen in real-world code.

Key Takeaways

- Hash tables deliver $O(1)$ average-case performance for insert, lookup, and delete.
- A hash function maps an arbitrary key to a fixed-size bucket index — no searching needed.

- Collisions are inevitable; C++ resolves them via chaining (linked list per bucket) or open addressing (linear probe).
- Use `std::unordered_map` for maximum speed; use `std::map` when sorted order is mandatory.
- Worst-case $O(n)$ exists in theory but is rarely triggered with C++'s built-in hash functions.